Exercise 2

Submission Deadline:
14.01.2026, 11:55 pm

General Information

This exercise will guide you through various iEMG (intramuscular) data postprocessing and analysis steps. We suggest using MATLAB to visualize, quantify and process the EMG data and the corresponding force signal during the exercises. Note that using Python is also allowed, although we will use MATLAB for the exemplary solution. If you have not installed MATLAB yet, click on the link below and take advantage of FAU's Campus-Wide License to install and use MATLAB on your computer: www.mathworks.com/products/matlab/student.html

In addition to the base MATLAB version, you will likely need the MATLAB signal processing toolbox (depends on the functions you want to use). You can install it in MATLAB under Home -> Add-Ons.

MATLAB provides a lot of built-in functions that can make your life easier during these exercises. It is always a good idea to check the **MATLAB documentation**. Don´t hesitate to also use **Google**, **an AI language model** or **write in the StudOn forum**. Also check the "Requirements and Tips" section on the last page of this document.

Submission:

1. **Your code** as a MATLAB (.m) or Python (.py) file. For MATLAB it's mandatory to use the R2025b version. For python version 3.13. The code must be structured according to the tasks and generate all plots and figures required by the tasks. The plots and figures that must be generated by your code are **written in this colour**. Further requirements for the plots can be found in the "Requirements and Tips" section of this document. **You have to submit the code file inside of the StudOn online test (see next point).**
2. **The online test**. It can be accessed in the StudOn course from the time of the exercise presentation until the submission deadline. **The test can only be submitted once.** The test contains both single/multiple choice questions and written text questions. For questions that require written text as an answer, 1-3 sentences or bullet points should be enough. The first "question" of the test is reserved for the code submission, where you have to upload your .m or .py file.

!

- **Only one code file may be submitted**. The code must be written in such a way that it runs completely with a single click on "Run" and fulfills all tasks. If the code does not run due to any errors, the submission will be marked as failed.
- This is an **individual exercise**. If the submitted code is identical to another submission, both submissions will be marked as failed.
- Passing the exercise is only possible if both, code AND online test are submitted.

Note that the interpretation of the exercise results and the physiological concepts behind these **are relevant for the exam.** Additionally, there will be **bonus points for each successfully passed exercise.** The exercise has a pass mark of 70 % of maximum points (24.5 pts).

The Dataset

For this exercise, you received a folder containing the following dataset: ***iEMG_contraction.mat***. The file contains EMG data acquired with a high-density intramuscular electrode that contains 16 electrodes (see figure 1). The electrode was placed inside the First Dorsal Interosseus (FDI) muscle of the right hand. The subject performed an isometric index finger abduction and tried to follow a similar force ramp as the one you saw in Exercise 1, with a plateau at 30 % of the MVC.

no movement

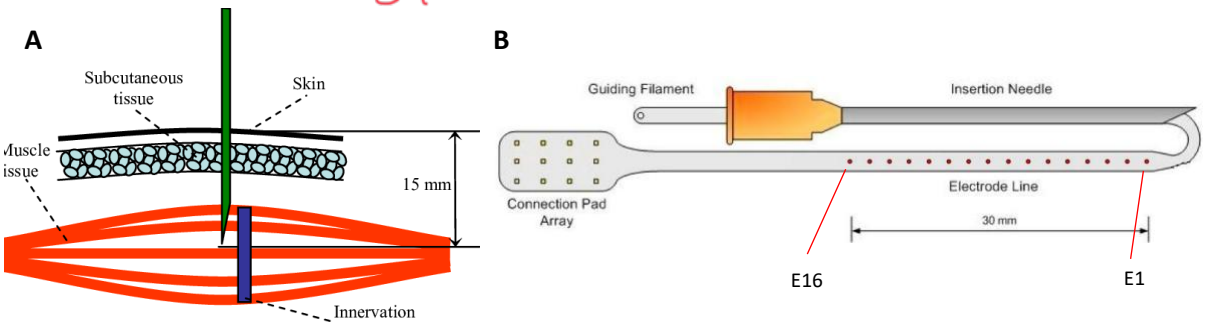

Figure 1: A: Schematic illustration of an electrode being inserted into the muscle. From Arslan et al. (2006) B: Intramuscular electrode containing 16 electrodes for iEMG recording.

Furthermore, the force signal (*force_signal*, in Newton) and the sampling frequency (*fsamp*, in Hz) are provided. You can also find the iEMG signals (*EMGSig*, in mV) in a cell with the shape of 8x2. Here the first column represents the signals of the first 8 electrodes, the second column the last 8 electrodes. In this case, for example *EMGSig{2,1}* would be electrode E2.

In addition to these variables, in this exercise you will need the motor unit action potential discharge timings (*MUPulses*). These timings, also called motor unit spikes, were obtained from the decomposition of the EMG data. As you know from the lectures, decomposition is based on blind source separation approaches to extract the single motor unit (MU) firings from the high-density EMG signals.

The variable *MUPulses* contains the exact samples, where the firings occurred (action potential discharge timings) for each motor unit during the muscle contraction. They are stored in a cell array (in this dataset, in total 17 motor units were found by the decomposition algorithm). That means for an exemplary motor unit #1 with action potential discharges at samples 5, 8 and 17: `MUPulses{1,1}=[5 8 17]`

The folder you received also contains the MATLAB functions **`plotSpikeRaster.m`** and **`spikeTriggeredAveraging.m`** that you will need during this exercise. Put these function files in your current working directory, so you can call them just like any native MATLAB function.

Task 1. MU Spike trains and discharge rate

Plotting the MU action potential discharge timings, as so-called **spike trains,** is a good way of visualizing the firing patterns of different motor units during isometric or kinetic contractions (e.g.: see Figure 2).

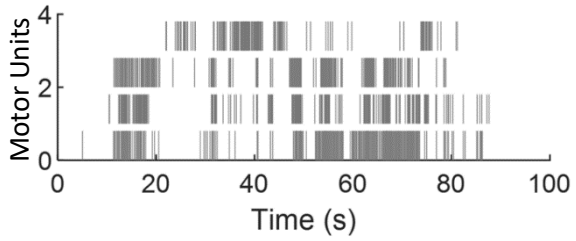

Figure 2: Exemplary MU Spike Trains. From: Del Vecchio et al. (2020).

For plotting the MU spike trains, you can use the provided MATLAB function `plotSpikeRaster(firingMatrix,'PlotType','vertline','VertSpikeHeight',0.9)`. The function requires a matrix `firingMatrix` with the so-called binary code of MU firings. Rows correspond to the MUs and columns to the samples (e.g. for motor unit data containing 5 MUs with a signal length of 2000 samples, the function requires a matrix of shape 5x2000), with "0" at the positions where the MU was not active, and "1" where the MU discharged an action potential (a firing occurred).

E.g., for an exemplary MU#1 with action potentials at samples 5, 8 and 17 and MU#2 at samples 3 and 12:

```
firingMatrix=[0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 1;
              0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0]
```

This matrix needs to be of data type **`logical`**. In MATLAB, you can convert an array or matrix of type `double` to an array of type `logical` with the command `logical(matrix)`.

1.1 **Plot the spike trains of all motor units (as shown in Figure 2) together with the force signal in N (use a separate y-axis for that) over time in the same plot. Use the y-axis for the number of the motor units. Choose a suitable visualization like in figure 2. (2 pts)**

1.2 **Create the same plot with all motor units and their firings again (together with the force signal in N), as in the previous task (1.1), but this time sort the motor units in ascending order according to their first firing time. This means that the motor unit that was the first to fire should be displayed at the bottom. (1 pt)** From now on, only work with the sorted order of motor units.

1.3 Another meaningful aspect to analyze is the **instantaneous discharge rate (IDR)** of individual motor units. The IDR represents the frequency between two consecutive firings and is typically expressed in Hz (1/s) or peak-per-second (pps) *(e.g.: two consecutive firings 50 ms apart = 20 Hz)*. Consequently, a motor unit will have exactly one fewer IDR data point compared to its number of firings, as each IDR value requires two consecutive firings. **Plot the IDRs of two different motor units (in Hz) of your choice over time (in seconds) together with the force signal in N (use a separate y-axis for that) (3 pts).** For the exact timing of each IDR, use the timing of the first firing in the pair of consecutive firings. Check figure 3 for an exemplary plot of the IDR of two MUs (these MUs are not from the data we gave to you), so you get an idea how it could look like.

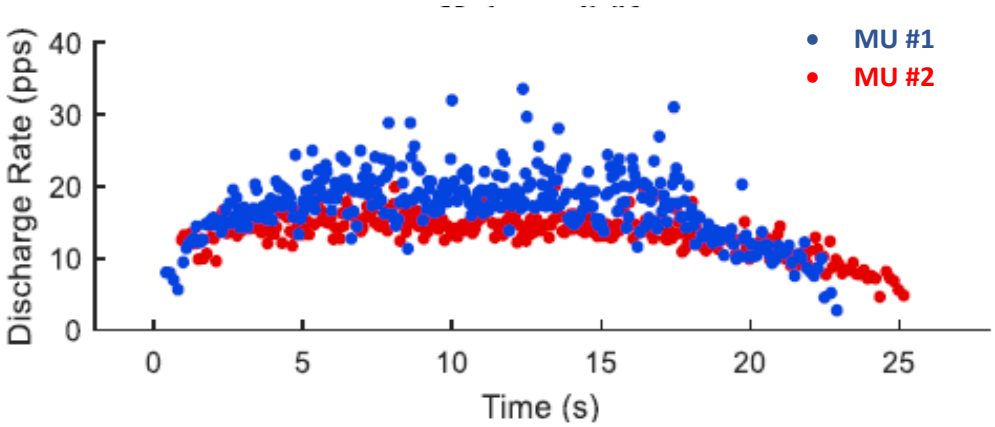

Figure 3: Instantaneous discharge rates of two exemplary motor units (force signal and title is missing here).

Task 2. Spike Triggered Averaging

Spike triggered averaging (STA) is a useful event detection technique that is often used in neural signal processing. STA is a measure to relate the spike trains to a continuous signal, in our case the raw EMG signal. It represents the average EMG signal shape inside a window around the times of spike occurrences and is therefore equivalent to the cross-correlation between the EMG signal and the spike train. As a result, the STA gives us the average **motor unit action potential (MUAP) shape** of a motor unit for each channel of our EMG data. Figure 4 illustrates the concept of STA with the spike train of an exemplary Motor Unit #12 (Note: this is not the MU #12 from the data we gave you).

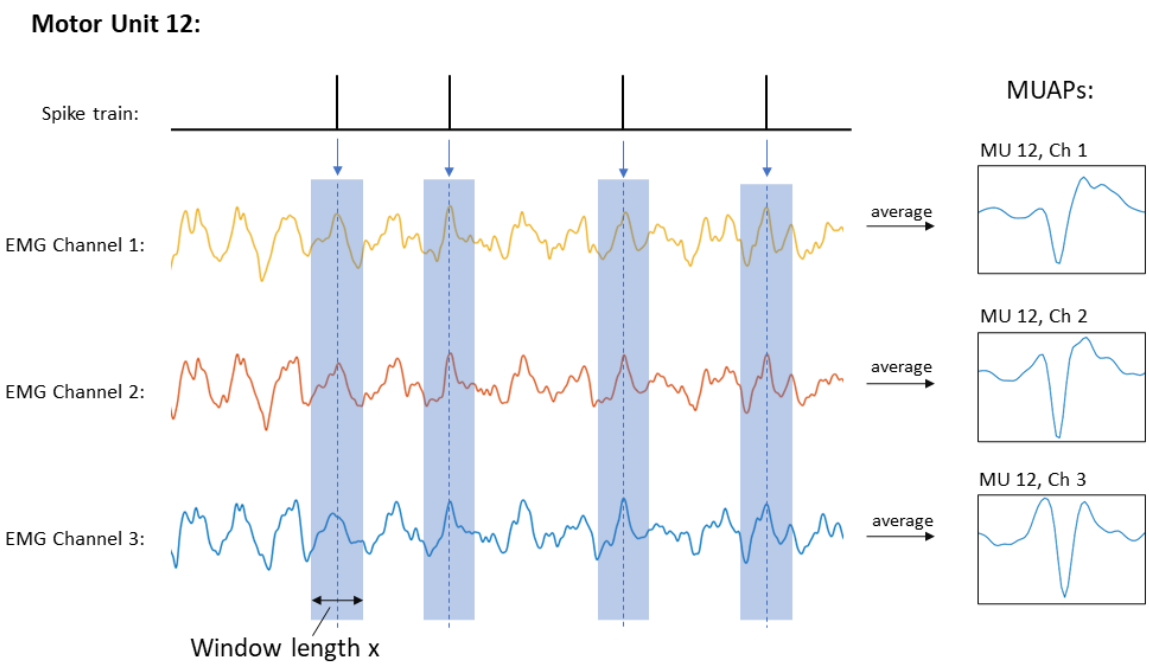

Figure 4: Concept of Spike triggered averaging. STA results in the MUAPs for each channel of the respective motor unit.

2.1 The provided `spikeTriggeredAveraging (SIG,MUPulses,STA_window,fsamp)` returns you the MUAP shapes of all motor units for each channel. Check the function's code for a description of its input/output parameters. **Plot the MUAP shapes of all 16 channels of one exemplary MU of your choice. Arrange the channel subfigures in the shape of the original EMG electrodes (16x1), with the first channel (electrode E1) at the bottom. Use the same vertical range for all channels. (3 pts)**

Task 3. Motor Unit Action Potential Amplitudes

From the lectures you should know that smaller motor units are usually recruited at lower forces, and bigger motor units at higher forces. Also, larger motor units produce larger action potentials than smaller motor units. Therefore, the recruitment threshold should show a positive correlation to the motor unit action potential amplitude.

3.1 Compute the maximum peak-to-peak value of the MUAP shapes of each MU (use the highest peak-to-peak value from all channels for each MU) to obtain the amplitude of the motor unit action potentials.

3.2 Visualize, if there is a dependency/correlation between the amplitude of the MUAP and the order of recruitment of the motor units (you sorted the MUs in task 1.2). **For this, generate a plot with the MUs in their physiological recruitment order on the horizontal axis and the amplitude of the MUAP on the vertical axis (scatter plot). Fit a linear regression model (regression line) to the scatter plot and report the R^2. (3 pts)**

Task 4. MU Locations

Using the MUAP shapes obtained from the STA, we can determine the spatial location of the motor units with respect to the intramuscular EMG electrodes (see Figure 5). The closer the motor unit is located to an electrode, the higher the amplitude of the MUAP in that specific channel will be.

4.1. Compute the root-mean-square (RMS) of the MUAP for each channel of each MU, resulting in 16 RMS values per MU, one for each channel. (No moving average is needed here, just the total RMS of each MUAP)

4.2. Arrange the RMS values for each MU in a vector according to the position of the electrodes, and **plot the resulting matrix of all motor units as a heatmap with 17 columns (one for each MU)** (MATLAB: `imagesc()`) **in one plot. (2 pts)** See Figure 4 for an exemplary heatmap of one motor unit. This way you can see the MUAP amplitude color-coded in the area of the EMG electrode grid, visualizing the corresponding motor unit location in the underlying muscle.

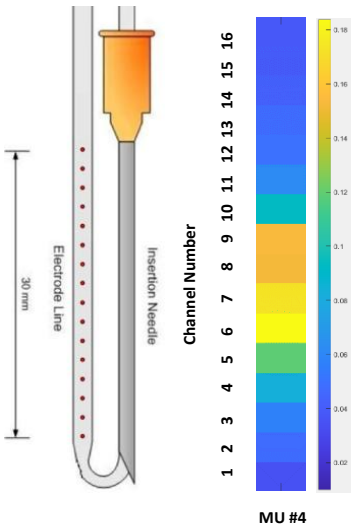

Figure 5: Left: iEMG electrode used for the acquisition of this data. Electrodes are aranged in a 16x1 matrix with the first electrode being the closest one to the tip of the needle. Right: Visualization of spatial location of one exemplary motor unit. One motor unit is visualised by one column. RMS values of MUAPs colorcoded according to its amplitude.

Requirements and Tips

- **Code Structure:** Keep your script well-structured and clear. For this, please create script sections by writing `%%`, followed by a blank space and a section title (please choose the section title according to the task). You can execute single script sections with the `Run Section` button in the toolbar or `Ctrl+Enter`, instead of always having to execute the entire script. (https://www.mathworks.com/help/matlab/matlab_prog/create-and-run-sections.html)

- **Plots and figures:** Each plot must have a clear title. The title should contain the respective task and a short description (e.g.: *Task 1.2: Force Signal of Isometric Ankle Dorsiflexion*). If a graph with axes is included, the axes must be labelled (with units and values).

  If you want to plot multiple related figures into one figure window, check out the `subplot` command. If you want to plot multiple plots into the same figure, use the `hold on` command. `yyaxis right` lets you plot multiple data series into one plot with independent y axes.

  (https://www.mathworks.com/help/matlab/ref/subplot.html)
  (https://www.mathworks.com/help/matlab/ref/hold.html)
  (https://www.mathworks.com/help/matlab/ref/yyaxis.html)

- **Custom Functions:** If you need to apply the same data processing steps to multiple sets of data, is it advised to write a custom function for these steps, that you can then pass the datasets to. In this exercise, please define your functions at the bottom of your code. Check https://www.mathworks.com/help/matlab/ref/function.html on how to define functions.

- If you do not know how to code a certain task, there is probably a **built-in MATLAB function** already available. Check the documentation, Google or the AI language model of your choice.